Permutations Complete DSA Notes + Leetcode 31 Next Permutation

1. What is a Permutation?

A permutation is an arrangement of elements in different possible orders.

Example:

Array:
[1,2,3]

Permutations:

[1,2,3]
[1,3,2]
[2,1,3]
[2,3,1]
[3,1,2]
[3,2,1]

Number of permutations:

n!

2. Permutation vs Combination

Permutation:
Order matters.

Example:
ABC and BAC are different.

Combination:
Order does not matter.

ABC and BAC are same.

3. Lexicographical Order

Permutations can be arranged like dictionary order.

Example:

123
132
213
231
312
321

Next permutation means:
Find the immediate bigger arrangement.

4. Leetcode 31 Problem Statement

Given an integer array nums.

Rearrange it into the next lexicographically greater permutation.

If impossible:
Return smallest permutation(sorted ascending).

Must modify array in-place.

Required:
O(n) time
O(1) space

5. Brute Force Approach

1. Generate all permutations.
2. Sort them.
3. Find current.
4. Return next.

Complexity:

Time:
O(n!*n)

Too slow.

6. Key Observation

Example:

1 2 3 6 5 4

From right side:

6 5 4 is decreasing.

The first smaller element:

3

This is the position where change happens.

7. Optimal Algorithm

Steps:

1. Find breakpoint

Find first index i from right:

nums[i] < nums[i+1]

2. Find just greater element than nums[i]
from right side.

3. Swap them.

4. Reverse remaining suffix.

Complexity:

Time:
O(n)

Space:
O(1)

Python - LC31 Next Permutation

```
        def nextPermutation(nums):
            n=len(nums)
```

```
    i=n-2


    # find breakpoint

    while i>=0 and nums[i]>=nums[i+1]:
        i-=1



    if i>=0:

        j=n-1


        while nums[j]<=nums[i]:
            j-=1


        nums[i],nums[j]=nums[j],nums[i]



    nums[i+1:]=reversed(nums[i+1:])


    return nums
```

C++ - LC31 Next Permutation

```
void nextPermutation(vector<int>& nums){

int n=nums.size();


int i=n-2;


// breakpoint

while(i>=0 && nums[i]>=nums[i+1])
    i--;



if(i>=0){

int j=n-1;


while(nums[j]<=nums[i])
    j--;


swap(nums[i],nums[j]);

}



// reverse suffix

reverse(
nums.begin()+i+1,
nums.end()
);

}
```

Java - LC31 Next Permutation

```
class Solution {

public void nextPermutation(int[] nums){

int n=nums.length;


int i=n-2;


// breakpoint

while(i>=0 && nums[i]>=nums[i+1])
    i--;



if(i>=0){

int j=n-1;


while(nums[j]<=nums[i])
    j--;


swap(nums,i,j);

}



reverse(nums,i+1,n-1);

}



void swap(int[] nums,int i,int j){

int temp=nums[i];
nums[i]=nums[j];
nums[j]=temp;

}



void reverse(int[] nums,int l,int r){

while(l<r){

swap(nums,l,r);

l++;
r--;

}

}

}
```

8. Generate All Permutations - Backtracking

Used in:

Leetcode 46 Permutations

Idea:

Choose
Explore
Undo(backtrack)

Complexity:

O(n!)

Python Backtracking

```
        def permute(nums):
            ans=[]

            def backtrack(start):
                if start==len(nums):
                    ans.append(nums[:])
                    return


                for i in range(start,len(nums)):
                    nums[start],nums[i]=nums[i],nums[start]
                    backtrack(start+1)
                    nums[start],nums[i]=nums[i],nums[start]


            backtrack(0)
            return ans
```

Permutation Patterns

Important Problems:

Easy/Medium:
- LC31 Next Permutation
- LC46 Permutations
- LC47 Permutations II
- LC60 Permutation Sequence

Advanced:
- Backtracking permutations
- Factorial number system
- Lexicographical ordering
- Heap's algorithm
- Bitmask permutations

Complexity Cheat Sheet

Generate all permutations:

Time:
O(n!*n)

Space:
O(n)

Next Permutation:

Time:
O(n)

Space:
O(1)

Recursive Backtracking depth:

O(n)